

**Министерство науки и высшего образования Российской Федерации
ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ
ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ ВЫСШЕГО ОБРАЗОВАНИЯ
НАЦИОНАЛЬНЫЙ ИССЛЕДОВАТЕЛЬСКИЙ УНИВЕРСИТЕТ ИТМО**

**ОТЧЕТ
ПО ЛАБОРАТОРНОЙ РАБОТЕ №3.2
«Создание таблиц базы данных PostgreSQL»**

По дисциплине «Проектирование и реализация баз данных»

Вариант 11

Обучающиеся: Востров Илья Анатольевич

Титор Матвей Андреевич

Факультет: ФПИИ

Группа: К3239

Направление подготовки: 09.03.03 Прикладная информатика

Образовательная программа: Мобильные и сетевые технологии

Преподаватель: Говорова Марина Михайловна

Санкт-Петербург

2025 г.

СОДЕРЖАНИЕ

СОДЕРЖАНИЕ	2
Цель работы	3
Практическое задание	3
Выполнение	4
Наименование БД	4
Инфологическая модель (IDEF1X)	4
Логическая модель базы данных (ERD)	5
Анализ ключей	6
Создание схемы базы данных	7
Создание таблиц	7
Ограничения данных	8
Primary Key	8
Unique	9
Check	9
Foreign Key	9
Заполнение таблиц	10
Проверка данных	10
Создание резервной копии	11
Восстановление базы данных	11
Dump базы данных	11
ВЫВОДЫ	12

Цель работы

Овладеть практическими навыками создания таблиц базы данных PostgreSQL 1X, заполнения их рабочими данными, резервного копирования и восстановления БД.

Практическое задание

1. Создать базу данных с использованием pgAdmin 4 (согласно индивидуальному заданию).
2. Создать схему в составе базы данных.
3. Создать таблицы базы данных.
4. Установить ограничения на данные: *Primary Key*, *Unique*, *Check*, *Foreign Key*.
5. Заполнить таблицы БД рабочими данными.
6. Создать резервную копию БД.
Указание:
Создать две резервные копии:
 - с расширением *CUSTOM* для восстановления БД;
 - с расширением для листинга (в отчете);
7. Восстановить БД

Выполнение

Наименование БД

База данных: AutoServiceDB

Схема: main_schema

Инфологическая модель (IDEF1X)

Схема инфологической модели в нотации IDEF1X представлена на рисунке 1

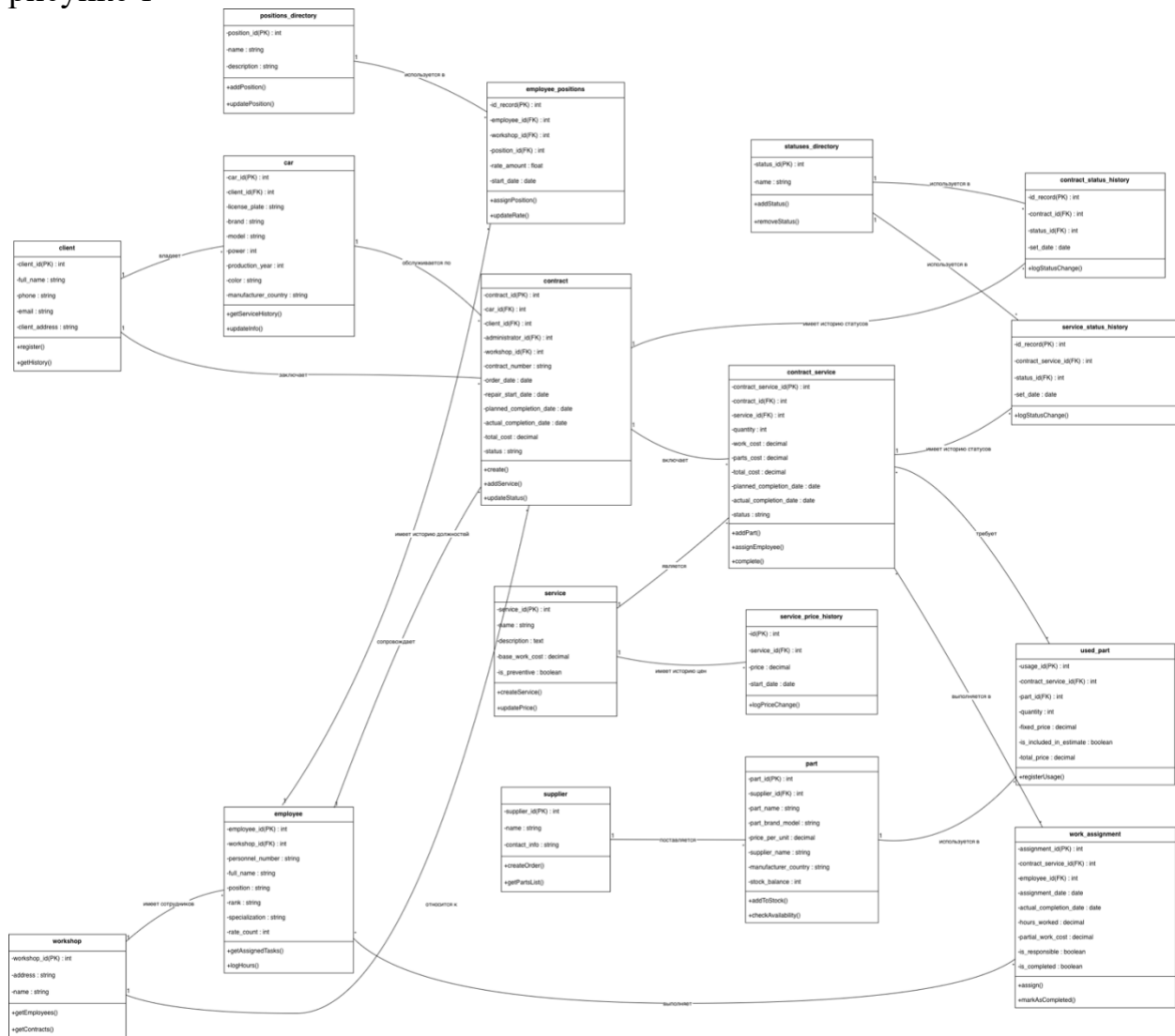


Рисунок 1 — Инфологическая модель базы данных (IDEF1X)

Логическая модель базы данных (ERD)

Схема инфологической модели данных представлена на рисунке 2:

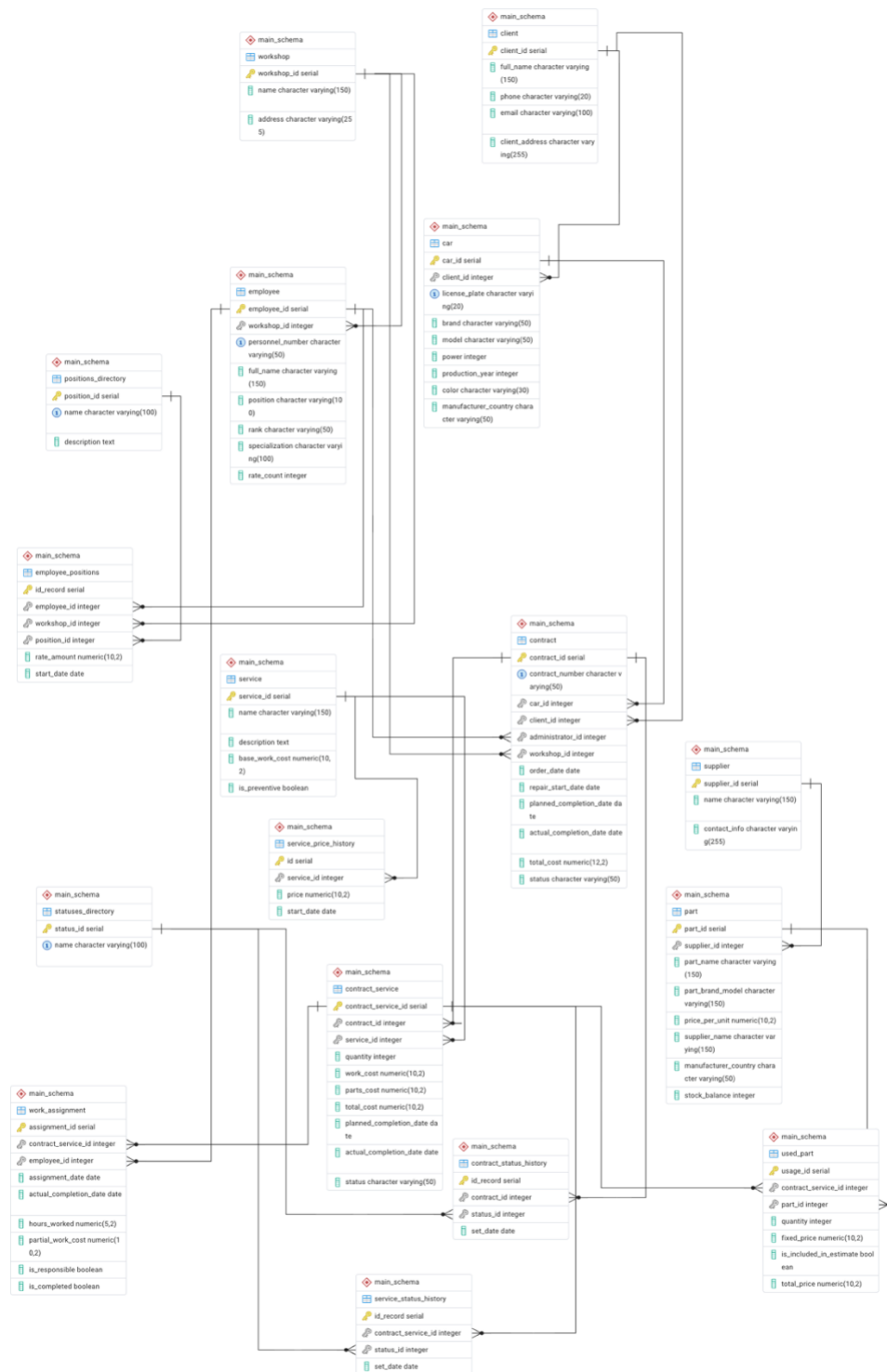


Рисунок 2 — Логическая модель базы данных (ERD)

Анализ ключей

В инфологической модели использовались составные ключи.

В процессе реализации базы данных они были заменены на искусственные ключи.

В каждой таблице в качестве первичного ключа используется целочисленное поле с автоинкрементом (sequence).

Примеры:

Таблица	Первичный ключ
client	client_id
car	car_id
contract	contract_id
employee	employee_id
service	service_id
part	part_id

Использование искусственных ключей упрощает структуру базы данных и ускоряет выполнение запросов.

Создание схемы базы данных

Для логической организации объектов базы данных была создана схема:

```
1 CREATE SCHEMA main_schema;
```

Листинг 1 — Создание схемы БД

Схема используется для группировки таблиц и других объектов базы данных.

Это позволяет структурировать данные и упростить администрирование.

Создание таблиц

Таблицы базы данных были созданы в соответствии с логической моделью.

Таблица **client** (Листинг 2) содержит информацию о клиентах автосервиса.

```
1 CREATE TABLE main_schema.client (  
2     client_id integer NOT NULL,  
3     full_name character varying(150) NOT NULL,  
4     phone character varying(20) NOT NULL,  
5     email character varying(100),  
6     client_address character varying(255)  
7 );
```

Листинг 2 — Создание таблицы client

Таблица **car** (Листинг 3) хранит информацию об автомобилях клиентов.

```
1 CREATE TABLE main_schema.car (  
2     car_id integer NOT NULL,  
3     client_id integer NOT NULL,  
4     license_plate character varying(20) NOT NULL,  
5     brand character varying(50) NOT NULL,  
6     model character varying(50) NOT NULL,  
7     power integer,  
8     production_year integer,  
9     color character varying(30),  
10    manufacturer_country character varying(50)  
11 );
```

Листинг 3 — Создание таблицы car

При создании таблиц учитывались:

- типы данных;
- обязательность полей (NOT NULL);
- логическая структура предметной области.

Ограничения данных

Для обеспечения целостности данных были заданы различные ограничения.

Primary Key

Первичный ключ обеспечивает уникальность записей в таблице (Листинг 4).

```
1 ALTER TABLE ONLY main_schema.client  
2 ADD CONSTRAINT client_pkey PRIMARY KEY (client_id);
```

Листинг 4 — Ограничение первичного ключа

Unique

Данное ограничение гарантирует, что регистрационный номер автомобиля уникален (Листинг 5).

```
1 ALTER TABLE ONLY main_schema.car
2 ADD CONSTRAINT car_license_plate_key UNIQUE (license_plate);
```

Листинг 5 — Ограничение уникальности

Check

Ограничения Check используются для контроля допустимых значений (Листинг 6).

```
1 CONSTRAINT car_power_check CHECK (power > 0)
2
3 CONSTRAINT car_production_year_check CHECK (
4     production_year >= 1885 AND production_year <= 2100
5 )
```

Листинг 6 — Ограничение Check

Foreign Key

Внешние ключи обеспечивают связь между таблицами и поддерживают ссылочную целостность (Листинг 7).

```
1 ALTER TABLE ONLY main_schema.car
2 ADD CONSTRAINT car_client_id_fkey
3 FOREIGN KEY (client_id)
4 REFERENCES main_schema.client(client_id)
5 ON DELETE CASCADE;
```

Листинг 7 — Ограничение внешнего ключа

Заполнение таблиц

Заполнение таблиц выполнялось с помощью SQL-запросов INSERT (Листинг 8-10).

```
1 INSERT INTO main_schema.client
2 VALUES (1, 'Иванов И.И.', '12345', 'a@a.ru', 'Москва');
```

Листинг 8 — Добавление клиента

```
1 INSERT INTO main_schema.car
2 VALUES (1, 1, 'A111AA', 'BMW', 'X5', 249, 2020, NULL, NULL);
```

Листинг 9 — Добавление автомобиля

```
1 INSERT INTO main_schema.service
2 VALUES (1, 'Замена масла', NULL, 1000.00, false);
```

Листинг 10 — Добавление услуги

Данные были добавлены с учетом всех ограничений, что подтверждает корректность структуры базы данных.

Проверка данных

Для проверки корректности заполнения таблиц использовались SELECT-запросы (Листинг 11):

```
1 SELECT * FROM main_schema.client;
2 SELECT * FROM main_schema.car;
3 SELECT * FROM main_schema.contract;
```

Листинг 11 — Проверка данных

Проверка показала, что данные успешно добавлены и соответствуют структуре базы данных.

Создание резервной копии

Резервное копирование базы данных выполнялось с помощью pgAdmin 4.

Были созданы две резервные копии:

1. формат **Custom** (для восстановления);
2. формат **Plain** (SQL-скрипт).

Использование двух форматов позволяет как восстановить базу данных, так и анализировать её структуру.

Восстановление базы данных

Восстановление базы данных выполнялось из ранее созданной резервной копии.

Также возможно восстановление через выполнение SQL-скрипта в Query Tool.

После восстановления была проверена корректность структуры и данных базы.

Dump базы данных

Был написан SQL-скрипт резервной копии базы данных **backup_plain.sql**.

Данный скрипт содержит:

- создание схемы;
- создание таблиц;
- ограничения;
- внешние ключи;
- команды вставки данных.

ВЫВОДЫ

В ходе выполнения лабораторной работы была разработана и реализована база данных автосервиса в СУБД PostgreSQL.

Были созданы таблицы, определены связи между ними и заданы ограничения целостности данных.

Также выполнено заполнение базы тестовыми данными.

Дополнительно были освоены навыки резервного копирования и восстановления базы данных.

Полученные знания позволяют эффективно проектировать и использовать реляционные базы данных в практических задачах.